

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn import metrics
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from IPython.display import display

# Load the dataset
try:
    df = pd.read_csv('Bank Churn Prediction.csv')
    print("First 5 rows of the dataset:")
    display(df.head())
except FileNotFoundError:
    print("Error: 'Bank Churn Prediction.csv' not found. Please ensure the file is in the correct location.")
    exit() # Exit if the file is not found
except Exception as e:
    print(f"An error occurred during data loading: {e}")
    exit() # Exit on other loading errors

# Data Exploration
print("\nColumn information:")
print(df.info())
print("\nDescriptive statistics:")
display(df.describe())
print("\nMissing values per column:")
display(df.isnull().sum())
print("\nNumber of duplicate rows:")
print(df.duplicated().sum())

# Analyze Feature Distributions (Histograms and Boxplots for numerical features)
# This section generates a pair of plots (histogram and boxplot) for each numerical column.
# Histograms show the frequency distribution of values, helping to identify skewness or multiple modes.
# Boxplots show the distribution's five-number summary (min, max, median, quartiles) and outliers.
numerical_features = ['CreditScore', 'Age', 'Tenure', 'Balance', 'NumOfProducts', 'EstimatedSalary']
for col in numerical_features:
    plt.figure(figsize=(10, 5)) # Adjusted figure size for better readability
    plt.subplot(1, 2, 1)
    sns.histplot(df[col], kde=True, bins=30) # Added bins for better histogram detail
    plt.title(f'Distribution of {col}', fontsize=14)
    plt.xlabel(col, fontsize=12)
    plt.ylabel('Frequency', fontsize=12)

    plt.subplot(1, 2, 2)
    sns.boxplot(y=df[col], palette='viridis') # Added palette for better aesthetics
    plt.title(f'Boxplot of {col}', fontsize=14)
    plt.ylabel(col, fontsize=12)

    plt.tight_layout() # Adjusts plot parameters for a tight layout
    plt.show()

# Analyze Feature Distributions (Bar charts or count plots for categorical features)
# This section generates a count plot for each categorical column.
# Count plots display the counts of observations in each category using bars.
# This helps in understanding the distribution of categorical variables.
categorical_features = ['Geography', 'Gender', 'HasCrCard', 'IsActiveMember', 'Exited']
for col in categorical_features:
    plt.figure(figsize=(7, 5)) # Adjusted figure size
    sns.countplot(x=df[col], palette='deep') # Added palette
    plt.title(f'Distribution of {col}', fontsize=14)
    plt.xlabel(col, fontsize=12)
    plt.ylabel('Count', fontsize=12)
    plt.xticks(rotation=0) # Ensure labels are not rotated unless necessary for long labels
    plt.show()

# Correlation Analysis
# This heatmap visualizes the correlation matrix of numerical features.
# It helps in identifying relationships between variables (e.g., strong positive or negative correlations).
# Values close to 1 or -1 indicate a strong linear relationship, while values near 0 indicate no linear relationship.
plt.figure(figsize=(12, 10))
try:
    correlation_matrix = df.corr(numeric_only=True)
    sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=.5) # Added fmt and linewidths for clarity
    plt.title('Correlation Matrix of Numerical Features', fontsize=16)
    plt.show()
except ValueError as e:

```

```

print(f"Error calculating correlation matrix: {e}")

# Target Variable Analysis
# This count plot shows the distribution of the target variable 'Exited'.
# It's important for understanding class imbalance, which can affect model training.
plt.figure(figsize=(6, 5))
sns.countplot(x='Exited', data=df, palette='pastel') # Added palette
plt.title('Distribution of Exited (Target Variable)', fontsize=14)
plt.xlabel('Exited (0 = No, 1 = Yes)', fontsize=12)
plt.ylabel('Count', fontsize=12)
plt.show()

# Data Cleaning - (No visualizations directly generated from this section, but it preprocesses data for modeling)
# Missing Value Imputation
for col in df.columns:
    if df[col].isnull().any():
        if pd.api.types.is_numeric_dtype(df[col]):
            df[col] = df[col].fillna(df[col].median())
        else:
            df[col] = df[col].fillna(df[col].mode()[0])

# Outlier Treatment (Capping at 95th percentile)
numerical_features = ['CreditScore', 'Age', 'Tenure', 'Balance', 'NumOfProducts', 'EstimatedSalary']
for col in numerical_features:
    if pd.api.types.is_numeric_dtype(df[col]):
        threshold = df[col].quantile(0.95)
        df[col] = np.where(df[col] > threshold, threshold, df[col])

# Categorical Feature Encoding
categorical_features_to_encode = ['Geography', 'Gender']
existing_categorical_features = [col for col in categorical_features_to_encode if col in df.columns]
if existing_categorical_features:
    df = pd.get_dummies(df, columns=existing_categorical_features, drop_first=True)

# Ensure 'HasCrCard' and 'IsActiveMember' are treated as categorical if not already
binary_categorical_features = ['HasCrCard', 'IsActiveMember']
for col in binary_categorical_features:
    if col in df.columns and not pd.api.types.is_numeric_dtype(df[col]):
        df[col] = df[col].astype('category').cat.codes

display(df.head())

# Data Preparation - (No visualizations directly generated from this section)
if 'Exited' in df.columns:
    y = df['Exited']
    X = df.drop(['Exited', 'RowNumber', 'CustomerId', 'Surname'], axis=1, errors='ignore')
else:
    print("Error: 'Exited' column not found in the DataFrame.")
    exit()

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Model Training - (No visualizations directly generated from this section)
if 'CreditScore' in X_train.columns:
    simple_linear_reg = LinearRegression()
    simple_linear_reg.fit(X_train[['CreditScore']], y_train)
    simple_linear_pred = simple_linear_reg.predict(X_test[['CreditScore']])
else:
    print("Warning: 'CreditScore' not found for Simple Linear Regression.")
    simple_linear_pred = np.array([])

multiple_linear_reg = LinearRegression()
multiple_linear_reg.fit(X_train, y_train)
multiple_linear_pred = multiple_linear_reg.predict(X_test)

log_reg = LogisticRegression(solver='liblinear', max_iter=2000)
log_reg.fit(X_train_scaled, y_train)
log_reg_pred = log_reg.predict(X_test_scaled)
log_reg_probs = log_reg.predict_proba(X_test_scaled)[: , 1]

decision_tree = DecisionTreeClassifier(random_state=42)
decision_tree.fit(X_train, y_train)
decision_tree_pred = decision_tree.predict(X_test)
decision_tree_probs = decision_tree.predict_proba(X_test)[: , 1]

random_forest = RandomForestClassifier(random_state=42)
random_forest.fit(X_train, y_train)
random_forest_pred = random_forest.predict(X_test)
random_forest_probs = random_forest.predict_proba(X_test)[: , 1]

```

```

knn = KNeighborsClassifier()
knn.fit(X_train_scaled, y_train)
knn_pred = knn.predict(X_test_scaled)
knn_probs = knn.predict_proba(X_test_scaled)[: , 1]

# Model Evaluation - (The plots are generated below this section)
print("\nRegression Model Evaluation:")
if simple_linear_pred.size > 0:
    print("Simple Linear Regression:")
    print(f"R-squared: {metrics.r2_score(y_test, simple_linear_pred):.2f}")
    print(f"MSE: {metrics.mean_squared_error(y_test, simple_linear_pred):.2f}")
    print(f"RMSE: {np.sqrt(metrics.mean_squared_error(y_test, simple_linear_pred)):.2f}")
else:
    print("Simple Linear Regression evaluation skipped due to missing 'CreditScore'.")

print("\nMultiple Linear Regression:")
print(f"R-squared: {metrics.r2_score(y_test, multiple_linear_pred):.2f}")
print(f"MSE: {metrics.mean_squared_error(y_test, multiple_linear_pred):.2f}")
print(f"RMSE: {np.sqrt(metrics.mean_squared_error(y_test, multiple_linear_pred)):.2f}")

print("\nClassification Model Evaluation:")

def evaluate_classification_model(model_name, y_true, y_pred, y_prob):
    print(f"\n{model_name}:")
    print(f"Accuracy: {metrics.accuracy_score(y_true, y_pred):.2f}")
    print(f"Precision: {metrics.precision_score(y_true, y_pred):.2f}")
    print(f"Recall: {metrics.recall_score(y_true, y_pred):.2f}")
    print(f"F1-score: {metrics.f1_score(y_true, y_pred):.2f}")
    fpr, tpr, thresholds = metrics.roc_curve(y_true, y_prob)
    auc = metrics.roc_auc_score(y_true, y_prob)
    print(f"AUC-ROC: {auc:.2f}")
    plt.plot(fpr, tpr, label=f'{model_name} (AUC = {auc:.2f})', linewidth=2) # Added linewidth

# Evaluate Classification Models
evaluate_classification_model("Logistic Regression", y_test, log_reg_pred, log_reg_probs)
evaluate_classification_model("Decision Tree", y_test, decision_tree_pred, decision_tree_probs)
evaluate_classification_model("Random Forest", y_test, random_forest_pred, random_forest_probs)
evaluate_classification_model("KNN", y_test, knn_pred, knn_probs)

# ROC Curve Visualization
# This plot compares the performance of different classification models.
# A curve closer to the top-left corner indicates better performance.
# The AUC (Area Under the Curve) value provides a single metric for overall performance,
# with 1 being perfect and 0.5 being random.
plt.figure(figsize=(8, 7))
plt.plot([0, 1], [0, 1], 'k--', color='gray', linestyle='--', linewidth=1) # Diagonal dashed line for reference
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate', fontsize=12)
plt.title('ROC Curves for Classification Models', fontsize=16)
plt.legend(loc='lower right', fontsize=10)
plt.grid(True, linestyle=':', alpha=0.7) # Added grid for readability
plt.show()

# Data Visualization (Additional visualizations)
# Bar chart for Accuracy scores
# This chart provides a direct comparison of the accuracy metric for each classification model.
# Higher bars indicate better accuracy.
accuracy_scores = {
    'Logistic Regression': metrics.accuracy_score(y_test, log_reg_pred),
    'Decision Tree': metrics.accuracy_score(y_test, decision_tree_pred),
    'Random Forest': metrics.accuracy_score(y_test, random_forest_pred),
    'KNN': metrics.accuracy_score(y_test, knn_pred)
}
models = list(accuracy_scores.keys())
accuracy = list(accuracy_scores.values())

plt.figure(figsize=(10, 6))
sns.barplot(x=models, y=accuracy, palette='viridis') # Using seaborn barplot with a palette
plt.xlabel("Models", fontsize=12)
plt.ylabel("Accuracy Score", fontsize=12)
plt.title("Accuracy Scores of Classification Models", fontsize=16)
plt.ylim(0.7, 0.9) # Adjusted y-axis limit for better comparison of model performance
plt.grid(axis='y', linestyle=':', alpha=0.7) # Add horizontal grid lines
plt.show()

# Bar chart for AUC-ROC scores
# Similar to accuracy, this chart compares the AUC-ROC scores, which is a good metric for
# classification models, especially with imbalanced datasets.
# Higher AUC-ROC values signify better model performance

```

```
# Higher AUC-ROC values signify better model performance.
auc_scores = {
    'Logistic Regression': metrics.roc_auc_score(y_test, log_reg_probs),
    'Decision Tree': metrics.roc_auc_score(y_test, decision_tree_probs),
    'Random Forest': metrics.roc_auc_score(y_test, random_forest_probs),
    'KNN': metrics.roc_auc_score(y_test, knn_probs)
}

models = list(auc_scores.keys())
auc = list(auc_scores.values())

plt.figure(figsize=(10, 6))
sns.barplot(x=models, y=auc, palette='magma') # Using seaborn barplot with a different palette
plt.xlabel("Models", fontsize=12)
plt.ylabel("AUC-ROC Score", fontsize=12)
plt.title("AUC-ROC Scores of Classification Models", fontsize=16)
plt.ylim(0.7, 0.9) # Adjusted y-axis limit for better comparison
plt.grid(axis='y', linestyle=':', alpha=0.7)
plt.show()
```

↩ First 5 rows of the dataset:

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember
0	1	15634602	Hargrave	619	France	Female	42	2	0.00	1	1	1
1	2	15647311	Hill	608	Spain	Female	41	1	83807.86	1	0	1
2	3	15619304	Onio	502	France	Female	42	8	159660.80	3	1	0
3	4	15701354	Boni	699	France	Female	39	1	0.00	2	0	0
4	5	15737888	Mitchell	850	Spain	Female	43	2	125510.82	1	1	1

```
Column information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   RowNumber             10000 non-null  int64
1   CustomerId            10000 non-null  int64
2   Surname               10000 non-null  object
3   CreditScore           10000 non-null  int64
4   Geography             10000 non-null  object
5   Gender               10000 non-null  object
6   Age                  10000 non-null  int64
7   Tenure               10000 non-null  int64
8   Balance              10000 non-null  float64
9   NumOfProducts        10000 non-null  int64
10  HasCrCard             10000 non-null  int64
11  IsActiveMember       10000 non-null  int64
12  EstimatedSalary      10000 non-null  float64
13  Exited               10000 non-null  int64
dtypes: float64(2), int64(9), object(3)
memory usage: 1.1+ MB
None
```

Descriptive statistics:

	RowNumber	CustomerId	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember
count	10000.00000	1.000000e+04	10000.000000	10000.000000	10000.000000	10000.000000	10000.000000	10000.00000	10000.000000
mean	5000.50000	1.569094e+07	650.528800	38.921800	5.012800	76485.889288	1.530200	0.70550	0.515100
std	2886.89568	7.193619e+04	96.653299	10.487806	2.892174	62397.405202	0.581654	0.45584	0.499797
min	1.00000	1.556570e+07	350.000000	18.000000	0.000000	0.000000	1.000000	0.00000	0.000000
25%	2500.75000	1.562853e+07	584.000000	32.000000	3.000000	0.000000	1.000000	0.00000	0.000000
50%	5000.50000	1.569074e+07	652.000000	37.000000	5.000000	97198.540000	1.000000	1.00000	1.000000
75%	7500.25000	1.575323e+07	718.000000	44.000000	7.000000	127644.240000	2.000000	1.00000	1.000000
max	10000.00000	1.581569e+07	850.000000	92.000000	10.000000	250898.090000	4.000000	1.00000	1.000000

Missing values per column:

	0
RowNumber	0
CustomerId	0
Surname	0
CreditScore	0
Geography	0
Gender	0

Age	0
Tenure	0
Balance	0
NumOfProducts	0
HasCrCard	0
IsActiveMember	0
EstimatedSalary	0